

TruAgent AI Assignment

Architecture Overview

The high-level flow of the system is:

Input → Intent Classification → Routing → Agent Execution → Response

User requests enter through the FastAPI endpoint, which passes the payload to the central Hub. The Hub acts as the main coordinator: it receives the user message, classifies the intent using the rule-based intent classifier, loads any relevant context, and routes the request to the appropriate workflow. The base agent then executes the required agents according to the workflow, with some agents running sequentially and others running in parallel. After execution, the Hub aggregates the agent outputs, generates the final response, and stores the updated context.

This follows a hub-and-spoke design: the Hub manages coordination, routing, and memory, while individual agents remain independent and focused on their own tasks.

Folder Map:

/api

Holds the FastAPI endpoints where the user sends the payload to the Hub.

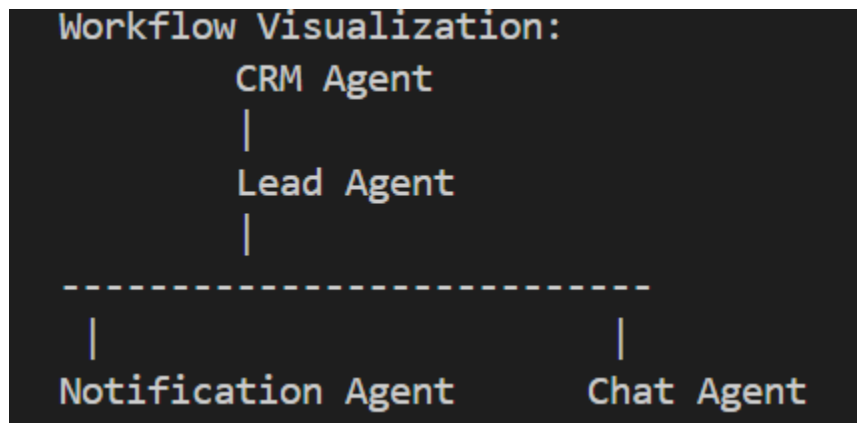
/orchestrator

Holds the Hub and supporting functions for intent classification, routing, and memory management. This acts as the brain of the system.

/agents

Holds the individual agents that perform the actual tasks.

Tech stack: Python, FastAPI, LLM-based intent classification (Gemini) with a rule-based fallback, JSON-based context handling, async agent execution.



How Orchestration Works

The orchestration layer coordinates the full request lifecycle: it classifies the user's intent, routes the request to the correct agents, loads prior context, executes the needed agent workflow, and persists the updated system state.

Intent Classification (`intent_classifier.py`)

The request first passes through intent classification, where the user message is scored against rule-based keywords related to sales and support. The classifier returns either "sales" or "support", and it includes a default path so the system never returns an empty or undefined intent.

Routing (`hub.py`)

Once the intent is known, `hub.py` routes the request to the correct execution path. A sales request is sent through the multi-agent sales pipeline, while a support request is handled by a single support agent.

Context Retrieval (`context_handler.py`)

Before any agent runs, the system retrieves the user's stored context from `context.json` using the `user_id`. If the user is new, `context_handler.py` creates a fresh record for them. This happens before execution so that agents can see prior state, such as whether the user already has an existing `contact_id`.

Agent Execution (`base.py`)

The sales pipeline is declared in `workflow.py` and rendered as a visualization, while the executor in `base.py` runs the equivalent pipeline: CRM first, then Lead, then chat and notify in parallel. The CRM agent's `create_contact()` step runs first because the rest of the sales process needs a real

contact record to attach work to. After that, the Lead agent runs `create_lead()`, because a lead depends on a contact already existing.

Once the lead has been created and scored, the chat and notification agents can run at the same time using `asyncio.gather`. That ordering-by-dependency is what makes it a DAG rather than a flat list.

Aggregation and Persistence (`hub.py`):

After the agents finish, `hub.py` assembles the final response object. This response includes the detected intent, the agents used, the routing reason, and the completed execution steps. Finally, the updated user context is written back to `context.json`, so future requests can reuse the new state.

Limitations

This implementation is intentionally scoped as an MVP: the goal is to demonstrate the end-to-end workflow clearly, while keeping several production concerns deliberately simplified.

The current context store is in-memory / JSON-based. A single JSON file is rewritten on every request, which is not safe under real concurrency because two simultaneous requests could overwrite each other; it also would not be shared across multiple server workers.

The workflow is declared and visualized, but not dynamically executed from the declaration. In `workflow.py`, the pipeline is represented as a workflow graph, but `base.py` runs a hardcoded equivalent rather than walking the workflow definition itself.

The agents are mocks rather than real integrations. CRM, Slack, and email actions are simulated with generated IDs and `asyncio.sleep`, which is sufficient for demonstrating orchestration but not for production side effects. The fix is to replace each mock agent with a real API client behind the same function signature, preserving the orchestration layer while making the actions real.

The endpoint also omits production hardening such as authentication, rate limiting, and retry/error recovery.

Demo Scenarios

```
{  
  "message": "I have an issue with my account",  
  "context": {  
    "source": "chat",  
    "user_id": "user-456"  
  }  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/agent/run' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"message": "I have an issue with my account", "context": {"source": "chat", "user_id": "user-456"}}'
```

Request URL

http://localhost:8000/agent/run

Server response

Code

Details

200

Response body

```
{  
  "intent": "support",  
  "routing": {  
    "agents": [  
      "support"  
    ],  
    "reason": "Detected a support intent, so the request is routed to the support agent only."  
  },  
  "steps": [  
    {  
      "agent": "support",  
      "status": "done",  
      "output": "Thank you for reaching out to support. A support agent will be with you shortly to assist you with your issue."  
    }  
  ],  
  "context": {  
    "contact_id": null,  
    "lead_id": null,  
    "last_interaction": 1780880384.2179585,  
    "interactions": 6,  
    "lead_score": 0,  
    "last_message": "I have an issue with my account"  
  }  
}
```

Download